

Operating Systems — Lab 2: Working with Processes

Varshita K and P Mahitha

Run: `./partN_partitioner.out file.txt NGTNIJGK 0 67108863 8388608` — 23 processes (15 partitioners, 8 searchers), pattern at byte 64 520 807.

Part I — one process, one range

`part1_searcher.cpp` reads the whole range into a buffer with a single `read` and slides the pattern along it with `memcmp`. It prints `[pid] found at <position>` and returns 1, otherwise `[pid] didn't find` and returns 0. That return value is what Part III later uses to decide the search is over.

Part II — the process tree

Each `part2_partitioner.out` is given a range. If the range is larger than the maximum chunk size, it halves the range and creates two more partitioners with `fork` and `exec`; otherwise it creates a single child running `part2_searcher.out`, which is the Part I code. A parent does no work of its own — it only calls `wait`. With 64 MiB and 8 MiB chunks that gives 15 partitioners and 8 searchers.

Every process is shown with its pid and the range it was responsible for. ★ marks the searcher that found the pattern.

```
[506] 0 - 67108863
|
+-- left [507] 0 - 33554431
| |
| +-- left [510] 0 - 16777215
| | +-- left [513] 0 - 8388607 --> searcher [520]
| | +-- right [516] 8388608 - 16777215 --> searcher [524]
| |
| +-- right [512] 16777216 - 33554431
| | +-- left [517] 16777216 - 25165823 --> searcher [525]
| | +-- right [521] 25165824 - 33554431 --> searcher [528]
|
+-- right [508] 33554432 - 67108863
|
| +-- left [509] 33554432 - 50331647
| | +-- left [514] 33554432 - 41943039 --> searcher [522]
| | +-- right [518] 41943040 - 50331647 --> searcher [526]
| |
| +-- right [511] 50331648 - 67108863
| | +-- left [515] 50331648 - 58720255 --> searcher [523]
| | +-- right [519] 58720256 - 67108863 --> searcher [527] ★ found at 64520807
```

Part II — order in which the processes were created

A process cannot exist before its parent has forked it, so the levels are fixed:

Level	Processes created	Created by
0	506 (root)	the shell
1	507, 508	506
2	509, 510, 511, 512	507 and 508
3	513, 514, 515, 516, 517, 518, 519, 521	509 – 512
4	520, 522, 523, 524, 525, 526, 527, 528 (the searchers)	513 – 521

The order observed in the run — Linux hands out pids sequentially, so the lower pid is always the older process:

```
506 -> 507 -> 508 -> 509 -> 510 -> 511 -> 512 -> 513 -> 514 -> 515
    -> 516 -> 517 -> 518 -> 519 -> 520 -> 521 -> 522 -> 523 -> 524
    -> 525 -> 526 -> 527 -> 528
```

Part II — order in which the processes terminated

Nothing is killed in Part II — **all 23 processes run to completion**. A partitioner exits once the last of its children has returned, so it is its **second** `child returned` line that marks its own exit, not its first:

```
520, 513, 522, 523, 514, 515, 525, 527★, 517, 519, 526, 511, 524, 518,
528, 516, 521, 509, 510, 512, 508, 507, 506
```

Part III — the process tree

A searcher that finds the pattern returns 1. A partitioner that reaps a child with exit status 1 sends `SIGTERM` to the other child and returns 1 itself, so the news travels up to the root. Each process also installs a `SIGTERM` handler that prints, forwards the same signal to its own children and exits — which collapses the entire subtree below it.

Same shape as Part II, with each pid carrying its own fate: `killed@N` means that process printed `received SIGTERM` at step N of the kill table below; `done` means it finished its chunk and printed `didn't find` on its own.

```

[533]                0 - 67108863
|
+-- left  [534] killed@8    0 - 33554431
|  |
|  +-- left  [536] killed@9    0 - 16777215
|  |  +-- left  [540]                0 - 8388607          --> searcher [546] done
|  |  +-- right [543]                8388608 - 16777215    --> searcher [551] done
|  |
|  +-- right [537] killed@9    16777216 - 33554431
|      +-- left  [541]                16777216 - 25165823  --> searcher [548] done
|      +-- right [545] killed@10    25165824 - 33554431  --> searcher [554] done
|
+-- right [535]                33554432 - 67108863
    |
    +-- left  [538] killed@6    33554432 - 50331647
    |  +-- left  [542] killed@6    33554432 - 41943039  --> searcher [550] done
    |  +-- right [547] killed@6    41943040 - 50331647  --> searcher [555] done
    |
    +-- right [539]                50331648 - 67108863
        +-- left  [544] killed@4    50331648 - 58720255  --> searcher [553] killed@4
        +-- right [549]                58720256 - 67108863 --> searcher [552] ★ found at 64520807

```

Only the winning path — 549, 539, 535 and the root 533 — exits normally, each returning 1. Of the eight searchers only 553 was actually cut off; an annotation on a partitioner line says nothing about the searcher below it.

Part III — order in which the processes were created

Part III changes nothing about how the tree is built:

Level	Processes created
0	533 (root)
1	534, 535
2	536, 537, 538, 539
3	540, 541, 542, 543, 544, 545, 547, 549
4	546, 548, 550, 551, 552, 553, 554, 555 (the searchers)

Part III — order in which the processes were killed

The kill propagates in one sweep from the successful leaf up to the root and back down the other branches:

Step	Event	Effect
1	[552] found at 64520807	the searcher exits with status 1
2	[549] searcher child returned	549 sees status 1, so it returns 1. It has no sibling to kill
3	[539] right child returned	539 sees status 1, sends SIGTERM to its left child 544, returns 1
4	[544] received SIGTERM	544 forwards to its searcher → [553] received SIGTERM
5	[535] right child returned	535 sees status 1, sends SIGTERM to its left child 538, returns 1
6	[538] received SIGTERM	538 forwards to 542 and 547, which print and forward in turn — but by then searchers 550 and 555 had already finished and exited, so nothing came of it
7	[533] right child returned	the root sees status 1, sends SIGTERM to its left child 534
8	[534] received SIGTERM	534 forwards to 536 and 537
9	[536] received SIGTERM, [537] received SIGTERM	536 had already reaped both children, so it forwards to nobody; 537 forwards to 545
10	[545] received SIGTERM	the last process to be taken down. Like 536 it had already reaped its child, so it too forwarded to nobody

Nine processes printed `received SIGTERM`: 534, 536, 537, 538, 542, 544, 545, 547, 553. The other six searchers had already finished their 8 MiB chunks before the signal reached them — on a multi-core machine they all finish at about the same time. `ps` after the run shows no surviving `part3_*` processes.

Observations

1. Creation is breadth-first, termination depth-first; the root dies last.
2. Part II wastes work: all eight searchers scan 8 MiB each, though only chunk 8 matters.
3. Killing has to be recursive — each handler forwards `SIGTERM` to its own children.
4. Success travels up as an exit status and back down as signals.
5. A match spanning a chunk boundary is found by neither; `pattern_length - 1` bytes of overlap would fix it.